

Course 50.050/50.550

Advanced Algorithms

Week 4 – Lecture L04.01



Outline of Lecture

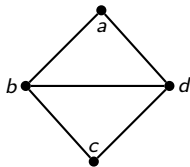
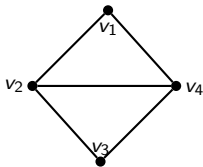
- ▶ Graph isomorphism
- ▶ Graph invariants
- ▶ Graph representations



Warm-up Example 1

Let $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ be two simple undirected graphs given as follows:

- ▶ $V_1 = \{v_1, v_2, v_3, v_4\}$, $E_1 = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}, \{v_1, v_4\}, \{v_2, v_4\}\}$.
- ▶ $V_2 = \{a, b, c, d\}$, $E_2 = \{\{a, b\}, \{b, c\}, \{c, d\}, \{a, d\}, \{b, d\}\}$.



- ▶ Are G_1 and G_2 the same graph?
 - ▶ No! G_1 and G_2 are technically not the same ordered pair.
 - ▶ After all $V_1 \neq V_2$. The vertices have different names!
 - ▶ Yet, intuitively, we “know” that the two graphs G_1 and G_2 have the “same graph structure”.

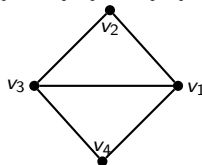
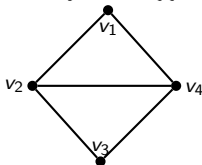
Question: What do we mean by “graph structure”?

- ▶ What do we mean when two “graph structures” are the same?
- ▶ Do the names of the vertices matter?

Warm-up Example 2

Let $G_3 = (V_3, E_3)$ and $G_4 = (V_4, E_4)$ be two simple undirected graphs given as follows:

- ▶ $V_3 = \{v_1, v_2, v_3, v_4\}$, $E_3 = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}, \{v_1, v_4\}, \{v_2, v_4\}\}$.
- ▶ $V_4 = \{v_1, v_2, v_3, v_4\}$, $E_4 = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}, \{v_1, v_4\}, \{v_1, v_3\}\}$.



- ▶ Are G_3 and G_4 the same graph?
 - ▶ No! G_3 and G_4 are technically not the same ordered pair.
 - ▶ Although V_3 and V_4 are the same set, the edge sets E_3 and E_4 are not the same. (e.g. $\{v_2, v_4\} \in E_3$, but $\{v_2, v_4\} \notin E_4$.)
 - ▶ Yet, intuitively, we “know” that the two graphs G_3 and G_4 have the “same graph structure”.

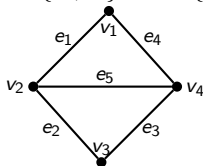
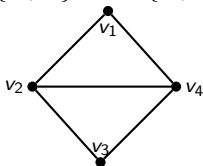
Question: What do we mean by “graph structure”?

- ▶ What do we mean when two “graph structures” are the same?
- ▶ How do we tell that the edges for the two “graph structures” match up?

Warm-up Example 3

Let $G_5 = (V_5, E_5)$ be a simple undirected graph, and $G_6 = (V_6, E_6, \phi_6)$ be an undirected graph, given as follows:

- ▶ $V_5 = \{v_1, v_2, v_3, v_4\}$, $E_5 = \{\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}, \{v_1, v_4\}, \{v_2, v_4\}\}$.
- ▶ $V_6 = \{v_1, v_2, v_3, v_4\}$, $E_6 = \{e_1, e_2, e_3, e_4, e_5\}$.
- ▶ $\phi_6 : E_6 \rightarrow \mathcal{P}(V_6)$ is the incidence function of graph G_6 , given as follows:
 - ▶ $e_1 \mapsto \{v_1, v_2\}$, $e_2 \mapsto \{v_2, v_3\}$, $e_3 \mapsto \{v_3, v_4\}$, $e_4 \mapsto \{v_1, v_4\}$, $e_5 \mapsto \{v_2, v_4\}$



- ▶ Are G_5 and G_6 the same graph?
 - ▶ No! G_5 and G_6 are technically not the same tuple.
 - ▶ G_5 is 2-tuple, while G_6 is a 3-tuple.
 - ▶ Yet, intuitively, we “know” that the two graphs G_5 and G_6 have the “same graph structure”.

Intuition: Whatever we define as “same graph structure”, it should agree with our intuition that all six graphs $G_1, G_2, G_3, G_4, G_5, G_6$ have the “same graph structure”.



Intuition for “same graph structure”

Recall: If G is a graph, then $V(G)$ always denotes the vertex set of G , and $E(G)$ always denotes the edge set of G .

Intuition: For two graphs G , G' to have the “same graph structure”:

- ▶ The vertices of G must correspond bijectively to the vertices of G' .
 - ▶ We can think of $V(G')$ as a “re-naming” of $V(G)$.
- ▶ The edges of G must correspond bijectively to the edges of G' .
 - ▶ We can think of $E(G')$ as a “re-naming” of $E(G)$.
- ▶ For any vertex v of G that is incident to some edge e of G , its corresponding vertex in $V(G')$ must also be incident to the corresponding edge in $E(G')$.
- ▶ Conversely, for any vertex v' of G' that is incident to some edge e' of G' , its corresponding vertex in $V(G)$ must also be incident to the corresponding edge in $E(G)$.

Note: The phrase “same graph structure” is informal!

- ▶ We have a formal/proper name for this idea of “sameness”.
 - ▶ We say that the graphs are **isomorphic**.



Graph isomorphism for undirected graphs

Definition: Let G, G' be **undirected** graphs. We say that G and G' are **isomorphic** graphs if there exist two **bijections** $f : V(G) \rightarrow V(G')$ and $g : E(G) \rightarrow E(G')$ such that for all $v \in V(G)$, $e \in E(G)$:

- ▶ The vertex v is incident to the edge e if and only if the vertex $f(v) \in V(G')$ is incident to the edge $g(e) \in E(G')$.
 - ▶ i.e. $v \in \phi(e) \Leftrightarrow f(v) \in \phi'(g(e))$.
 - ▶ Here, ϕ, ϕ' denote the incidence functions of G, G' , respectively.
 - ▶ **Note:** For a simple graph, whether it is defined as a pair (V, E) or as a 3-tuple (V, E, ϕ) , the notion of “incidence function” still makes sense.

Notation: We write $G \cong G'$ to mean G and G' are isomorphic graphs.

▶ **Warning:** Some authors use $G \simeq G'$, which is bad/non-standard notation. The ordered pair of bijections (f, g) is called a **graph isomorphism** from G to G' (or a **graph isomorphism** between G and G').

- ▶ We typically give a name to the ordered pair (f, g) , say $\varphi := (f, g)$, and view φ as a map from $(V(G), E(G))$ to $(V(G'), E(G'))$.

Only if you are interested: The words “isomorphic” and “isomorphism” have precise meanings in mathematics, and are used in many branches of mathematics, most notably in algebra. An isomorphism is a **structure-preserving map between structures of the same “type”**. We could have isomorphisms between graphs, isomorphisms between vector spaces, isomorphisms between algebraic groups, etc.

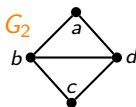
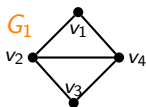


Examples of graph isomorphisms (undirected graphs)

Previously, we defined six graphs $G_1, G_2, G_3, G_4, G_5, G_6$.

- ▶ Graphs G_1, G_3, G_5 have the exact same vertex set and the exact same edge set, so these graphs are equal, i.e. $G_1 = G_3 = G_5$.
- ▶ Equality of graphs implies graph isomorphism, so $G_1 \cong G_3 \cong G_5$.

Example: How to show that $G_1 \cong G_2$.



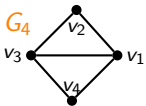
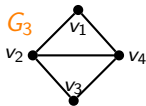
Bijection $f : V(G_1) \rightarrow V(G_2)$

- ▶ $v_1 \mapsto a, v_2 \mapsto b, v_3 \mapsto c, v_4 \mapsto d$

Bijection $g : E(G_1) \rightarrow E(G_2)$

- ▶ $\{v_i, v_j\} \mapsto \{f(v_i), f(v_j)\}$.

Example: How to show that $G_3 \cong G_4$.



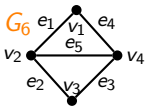
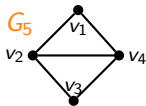
Bijection $f : V(G_3) \rightarrow V(G_4)$

- ▶ $v_1 \mapsto v_2, v_2 \mapsto v_3, v_3 \mapsto v_4, v_4 \mapsto v_1$

Bijection $g : E(G_3) \rightarrow E(G_4)$

- ▶ $\{v_i, v_j\} \mapsto \{f(v_i), f(v_j)\}$.

Example: How to show that $G_5 \cong G_6$.



Bijection $f : V(G_5) \rightarrow V(G_5)$

- ▶ $v_1 \mapsto v_1, v_2 \mapsto v_2, v_3 \mapsto v_3, v_4 \mapsto v_4$

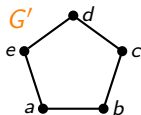
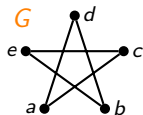
Bijection $g : E(G_5) \rightarrow E(G_6)$

- ▶ $\{v_1, v_2\} \mapsto e_1, \{v_2, v_3\} \mapsto e_2,$
 $\{v_3, v_4\} \mapsto e_3, \{v_1, v_4\} \mapsto e_4, \{v_2, v_4\} \mapsto e_5.$



More examples of graph isomorphisms

Example: Graphs G , G' depicted below are isomorphic.



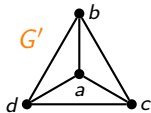
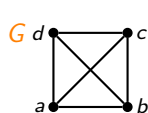
Bijection $f : V(G) \rightarrow V(G')$

► $a \mapsto a, b \mapsto d, c \mapsto b, d \mapsto e, e \mapsto c$

Bijection $g : E(G) \rightarrow E(G')$

► $\{u, v\} \mapsto \{f(u), f(v)\}.$

Example: Graphs G , G' depicted below are isomorphic.



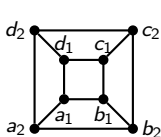
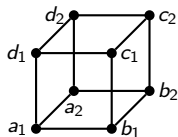
Bijection $f : V(G) \rightarrow V(G')$

► $a \mapsto a, b \mapsto b, c \mapsto c, d \mapsto d$

Bijection $g : E(G) \rightarrow E(G')$

► $\{u, v\} \mapsto \{f(u), f(v)\}.$

Example: Graphs G , G' depicted below are isomorphic.



Bijection $f : V(G) \rightarrow V(G')$

► $a_i \mapsto a_i, b_i \mapsto b_i, c_i \mapsto c_i, d_i \mapsto d_i$

Bijection $g : E(G) \rightarrow E(G')$

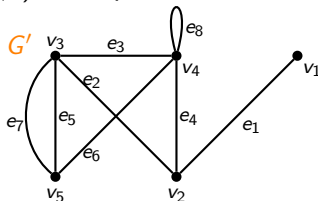
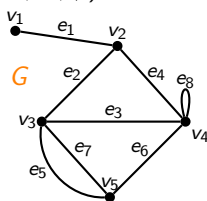
► $\{u, v\} \mapsto \{f(u), f(v)\}.$

Simpler definition: Let $G = (V, E)$, $G' = (V', E')$ be **simple** undirected graphs. We say that G and G' are **isomorphic** graphs if there exists a **bijection** $f : V \rightarrow V'$ such that for all distinct vertices $u, v \in V$, we have $\{u, v\} \in E$ if and only if $\{f(u), f(v)\} \in E'$.



Example: Isomorphism of non-simple undirected graphs

$G = (V, E, \phi)$ and $G' = (V', E', \phi')$, as depicted below, are isomorphic.



- ▶ Bijection $f : V \rightarrow V'$ given by $v_i \mapsto v_i$ for all $1 \leq i \leq 8$.
- ▶ Bijection $g : E \rightarrow E'$ given by $e_i \mapsto e_i$ for all $1 \leq i \leq 8$.
- ▶ We check that ϕ and ϕ' are identical incidence functions.

Intuition:

- ▶ If there is exactly 1 loop in G , then there is exactly 1 loop in G' .
 - ▶ Number of loops is an invariant for all graphs isomorphic to G .
- ▶ If G has multiple edges, then G' also has multiple edges.
 - ▶ The property “has multiple edges” is an invariant for all graphs isomorphic to G .
- ▶ If $\max\{\deg_G(v) | v \in V\} = 5$, then $\max\{\deg_{G'}(v) | v \in V'\} = 5$.
 - ▶ The maximum degree of the vertices is an invariant for all graphs isomorphic to G .



Graph isomorphism for directed graphs

Definition: Let G and G' be **directed** graphs, and let ϕ and ϕ' be their respective incidence functions. We say that G and G' are **isomorphic** graphs if there exist two **bijections** $f : V(G) \rightarrow V(G')$ and $g : E(G) \rightarrow E(G')$, such that for all $u, v \in V(G)$, $e \in E(G)$:

- ▶ The vertices u, v are the head and tail, respectively, of the edge e if and only if vertices $f(u), f(v)$ are the head and tail, respectively, of the edge $g(e)$.
- ▶ i.e. $\phi(e) = (u, v) \Leftrightarrow \phi'(g(e)) = (f(u), f(v))$.

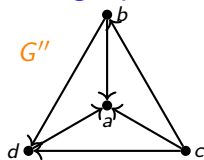
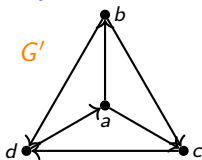
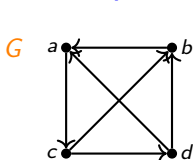
Same notation: $G \cong G'$ means G and G' are isomorphic graphs.

- ▶ **Same warning:** Some authors use the bad notation $G \simeq G'$.

Same terminology: (f, g) is called a **graph isomorphism** from G to G' (or a **graph isomorphism** between G and G').

Intuition: Two directed graphs are isomorphic if their respective “underlying undirected graphs” (i.e. “ignoring” directions) are isomorphic, and the “directions” of the directed edges match up.

Examples: Isomorphisms of directed graphs



Note: $G \cong G'$.

- ▶ Bijection $f : V(G) \rightarrow V(G')$ given by $a \mapsto c, b \mapsto d, c \mapsto a, d \mapsto b$.
- ▶ Bijection $g : E(G) \rightarrow E(G')$ given by $\{u, v\} \mapsto \{f(u), f(v)\}$.

Question: Why is G' not isomorphic to G'' ?

- ▶ G'' has a vertex a whose in-degree is $\deg_{G''}^{-}(a) = 3$.
 - ▶ The property “has a vertex with in-degree 3” is an invariant for all graphs isomorphic to G'' .
- ▶ Yet, all of the vertices x in G' have in-degrees $\deg_{G'}^{-}(x) \neq 3$.

Simpler definition: Let $G = (V, E)$, $G' = (V', E')$ be **simple** directed graphs. We say that G and G' are **isomorphic** graphs if there exists a **bijection** $f : V \rightarrow V'$ such that for all distinct vertices $u, v \in V$, we have $(u, v) \in E$ if and only if $(f(u), f(v)) \in E'$.

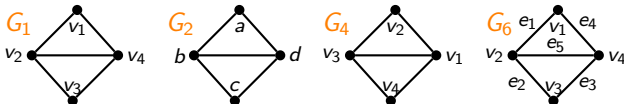


Graph isomorphism classes

Definition: Let G be a graph. The set of all graphs that are isomorphic to G is called the **isomorphism class** of G .

- ▶ We call this set of graphs a **graph isomorphism class**, or if the context is clear, an **isomorphism class**.
- ▶ Two graphs are isomorphic if and only if they are in the same graph isomorphism class.

Example: The graphs G_1 , G_2 , G_4 , G_6 , as depicted below, are not the exact same graph, but they are in the same isomorphism class.



Technicality: When we introduced various special classes of graphs in L03.02 (i.e. K_n , $K_{n,m}$, P_n , C_n), we are technically defining entire isomorphism classes of graphs.

- ▶ e.g. for $n \in \mathbb{Z}^+$, if we define the set $S := \{1, 2, \dots, n\}$, then K_n refers to any graph in the isomorphism class of the simple graph $(S, \binom{S}{2})$.

Graph invariants

Recall: (Week 2 cohort class) We introduced an **invariant** to be a fixed proposition that is initially true, and remains true after any sequence of “moves”, independent of the choice of such a sequence.

- ▶ More formally, and more generally, an **invariant** is a k -ary predicate P (for some $k \in \mathbb{N}$), such that for all k -tuples $(a_1, \dots, a_k)$ in the domain of discourse, the proposition $P(a_1, \dots, a_k)$ is true.

▶ **Recall:** A proposition is a 0-ary predicate.

Definition: A **graph invariant** is a 1-ary predicate P concerning a graph property or a value associated to a graph, such that if the proposition $P(G)$ is true for some graph G , then the proposition $P(G')$ is true for **all** graphs G' in the isomorphism class of G .

- ▶ **Intuition:** A graph invariant is a property or a value that does not depend on the names of the vertices or edges, and does not depend on any particular drawing of the graph.
- ▶ **Intuition:** If a graph invariant is initially true when we start with some graph in an isomorphism class, then the invariant remains true as we “move” to any other graph in this isomorphism class.



Some examples of graph invariants

- ▶ Number of vertices
 - ▶ e.g. if graph G satisfies $|V(G)| = 10$, then **all** graphs G' in the isomorphism class of G must satisfy $|V(G')| = 10$.
- ▶ Number of edges
 - ▶ e.g. if graph G satisfies $|E(G)| = 20$, then **all** graphs G' in the isomorphism class of G must satisfy $|E(G')| = 20$.
- ▶ Simple property
 - ▶ e.g. if graph G is simple, then **all** graphs $G' \cong G$ must be simple.
- ▶ Connected property
 - ▶ e.g. if graph G is connected, then **all** graphs G' in the isomorphism class of G must be connected.
- ▶ For any fixed $k \in \mathbb{N}$, the number of vertices of degree k .
 - ▶ e.g. if an undirected graph G has exactly 4 vertices of degree 5, then **all** graphs G' in the isomorphism class of G must have exactly 4 vertices of degree 5.
- ▶ Maximum degree
 - ▶ e.g. if the maximum degree of the vertices in an undirected graph G is 7, then for **all** graphs G' in the isomorphism class of G , the maximum degree of the vertices in G' must be 7.

Note: We will be learning numerous more graph invariants.



How to determine if two graphs are isomorphic?

Problem: Given two graphs G, G' , determine if $G \cong G'$ or $G \not\cong G'$.

- ▶ This problem is known as the **graph isomorphism problem**.

Very useful method: If we can find a graph invariant P such that $P(G)$ is true while $P(G')$ is false, then we can conclude that $G \not\cong G'$.

- ▶ e.g. let P represents “connectedness”. If G is connected (i.e. $P(G)$ is true) and G' is not connected (i.e. $P(G')$ is false), then $G \not\cong G'$.

Note: Usually, it is easy to verify that two graphs are not isomorphic by finding a suitable graph invariant for which both graphs differ.

- ▶ However, verifying that two graphs are isomorphic is much harder (especially for larger graphs).
- ▶ Even if G, G' have multiple same graph invariants, we still have to use the definition of “isomorphic” and check all possible permutations of the vertex set and edge set, to verify $G \cong G'$.

Only if you are interested: For a long time, it was a famous unsolved problem whether the graph isomorphism problem is solvable in polynomial time.

Latest Progress: In 2015, Hungarian mathematician László Babai announced a proof that the graph isomorphism problem is solvable in **quasi-polynomial time**.

- ▶ As of 2024, it seems that the proof is still being checked by other experts.

Graph representations

Definition: A **graph representation** is a data structure for storing the information of a graph $G = (V, E, \phi)$ (or the isomorphism class of G).

- ▶ **Recall:** A **data structure** is a format to **store and organize** data in a machine, in order to facilitate access and modification.

Note: There are four common graph representations:

- ▶ Incidence matrix (for arbitrary graphs)
- ▶ Incidence list (for arbitrary graphs)
 - ▶ Possible to implement as an array of linked lists.
 - ▶ Possible to implement as a hash table whose slots are linked lists.
- ▶ Adjacency matrix (for simple graphs)
- ▶ Adjacency list (for simple graphs)
 - ▶ Possible to implement as an array of linked lists.
 - ▶ Possible to implement as a hash table whose slots are linked lists.

Things to take note of when encoding graphs:

- ▶ Are the names of the vertices and edges important?
- ▶ What do you want to do with the graph after storing it?
- ▶ Does the graph change over time?

Incidence matrix

Definition: Let $G = (V, E, \phi)$ be a graph. Suppose $V = \{v_1, \dots, v_n\}$, and $E = \{e_1, \dots, e_m\}$. The **incidence matrix** of G is an n -by- m matrix $M = [m_{i,j}]$, whose rows and columns correspond to vertices and edges respectively, and whose (i, j) -th entry $m_{i,j}$ is defined as follows:

- ▶ (If G is undirected): $m_{i,j} = \begin{cases} 1, & \text{if } v_i \text{ is incident to } e_j; \\ 0, & \text{otherwise.} \end{cases}$
- ▶ (If G is directed):
$$m_{i,j} = \begin{cases} 1, & \text{if } e_j \text{ is a non-loop and } v_i \text{ is the head of } e_j; \\ -1, & \text{if } e_j \text{ is a non-loop and } v_i \text{ is the tail of } e_j; \\ 0, & \text{otherwise.} \end{cases}$$

Warning! Some authors instead define $m_{i,j} = 1$ if v_i is the tail of e_j , and $m_{i,j} = -1$ if v_i is the head of e_j .

- ▶ **Note:** For 50.050, we shall use the convention that $m_{i,j} = 1$ if v_i is the head of e_j , $m_{i,j} = -1$ if v_i is the tail of e_j .

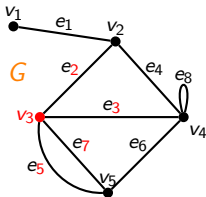
Question: Can you see how the information about the incidence function ϕ is stored in the incidence matrix of G ?



Basic properties of incidence matrices

Let $G = (V, E, \phi)$ be an **undirected** graph. Let $V = \{v_1, \dots, v_n\}$, let $E = \{e_1, \dots, e_m\}$, and let $M = [m_{i,j}]$ be the incidence matrix of G .

Example:



Incidence matrix of G :

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

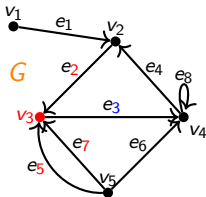
- ▶ The column sum of the j -th column of M equals 2 if e_j is a non-loop, and equals 1 if e_j is a loop.
 - ▶ Here, **column sum** refers to the sum of the entries in a column.
 - ▶ **Note:** In the example above, the column sum of the 8-th column equals 1.
- ▶ If G has no loops, then the row sum of the i -th row of M equals $\deg(v_i)$.
 - ▶ Here, **row sum** refers to the sum of the entries in a row.
 - ▶ For general undirected graphs with loops, the row sum of the i -th row of M equals $\deg(v_i) - |\{\text{loops incident to } v_i\}|$.
 - ▶ **Recall:** A loop contributes +2 to the degree.



Basic properties of incidence matrices (continued)

Let $G = (V, E, \phi)$ be a **directed** graph. Let $V = \{v_1, \dots, v_n\}$, let $E = \{e_1, \dots, e_m\}$, and let $M = [m_{i,j}]$ be the incidence matrix of G .

Example:



Incidence matrix of G :

$$\begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & \textcolor{red}{1} & \textcolor{blue}{-1} & 0 & \textcolor{red}{1} & 0 & \textcolor{red}{1} & 0 \\ 0 & 0 & 1 & -1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & -1 & -1 & 0 \end{bmatrix}$$

- ▶ The column sum of every column of M always equals 0.
 - ▶ e_j is a loop if and only if the j -th column of M is the zero column vector.
- ▶ The number of 1's in the i -th row equals the number of non-loop edges with v_i as head, and the number of -1 's in the i -th row equals the number of non-loop edges with v_i as tail.

Note: In contrast to the undirected case, the incidence matrix of a directed graph with loops does NOT store information about the vertex that each loop is incident to.

Incidence list

Definition: Let $G = (V, E, \phi)$ be a graph. Suppose $V = \{v_1, \dots, v_n\}$, and $E = \{e_1, \dots, e_m\}$. The **incidence list** of G is an array $A[1..n]$ of length n , whose entries are linked lists, such that:

- ▶ (If G is undirected): $A[i]$ is a linked list whose elements are pointers to edges e_j incident to v_i .
- ▶ (If G is directed): $A[i]$ is a linked list whose elements are pointers to edges e_j with v_i as the tail (i.e. initial vertex).

Note: This array of linked lists can be implemented as a hash table, where each slot is a linked list (i.e. collisions are resolved by chaining).

Possible Python implementation: Python dictionary A with key-value pairs `name_i : list_i`, where `name_i` is a character string representing the name for vertex v_i , and `list_i` is the linked list associated to v_i , which is accessed via $A[\text{name}_i]$.

Intuition: Store vertices, and for each vertex, store its incident edges.

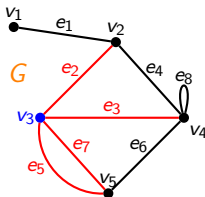
Advantages of incidence list as a graph representation:

- ▶ Very useful if you want to incrementally insert edges.
 - ▶ e.g. if the graph represents a social network that varies over time.
 - ▶ Insertion of new edges in each linked list has complexity $\Theta(1)$.
- ▶ Since edges are stored as pointers in linked lists, we can store multiple attribute values for edges (e.g. edge weights)



Examples of incidence lists

Undirected case:

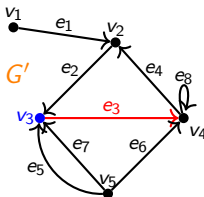


Incidence list A of G :

$A[v_1]$	→
$A[v_2]$	→ → →
$A[v_3]$	→ → → →
$A[v_4]$	→ → → →
$A[v_5]$	→ → →

- ▶ Every non-loop appears twice in A ; every loop appears once in A .
- ▶ Number of elements in linked list $A[i]$ is $\deg(v_i) - |\{\text{loops incident to } v_i\}|$.

Directed case:



Incidence list A of G' :

$A[v_1]$	→
$A[v_2]$	→
$A[v_3]$	→
$A[v_4]$	→ →
$A[v_5]$	→ → →

- ▶ Every edge appears once in A .
- ▶ Number of elements in linked list $A[i]$ is out-degree $\deg^+(v_i)$.



Adjacency matrix

Definition: Let $G = (V, E)$ be a **simple** graph, and suppose that $V = \{v_1, \dots, v_n\}$. The **adjacency matrix** of G is an n -by- n square matrix $A = [a_{i,j}]$, whose (i,j) -th entry $a_{i,j}$ is defined as follows:

- ▶ (If G is undirected): $a_{i,j} = \begin{cases} 1, & \text{if } v_i \text{ is adjacent to } v_j; \\ 0, & \text{otherwise.} \end{cases}$
- ▶ (If G is directed): $a_{i,j} = \begin{cases} 1, & \text{if } (v_i, v_j) \in E; \\ 0, & \text{otherwise.} \end{cases}$

Basic properties:

- ▶ Since G has no loops, the diagonal entries $a_{i,i}$ of A are all zeros.
- ▶ The adjacency matrix of an **undirected** graph is symmetric.
 - ▶ $A = [a_{i,j}]$ is called **symmetric** if $a_{i,j} = a_{j,i}$ for all $1 \leq i, j \leq n$.

Why are adjacency matrices so important?

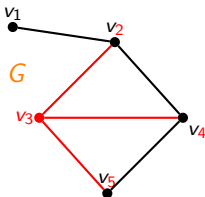
- ▶ Several graph invariants can be computed from the adjacency matrix. *(More in this week's cohort class and in later weeks.)*

Warning! Some authors extend the definition of adjacency matrices to non-simple graphs. Although we could certainly extend the definition, it has the disadvantage that some theorems involving adjacency matrices are no longer applicable in the non-simple case.



Examples of adjacency matrices

Undirected case:

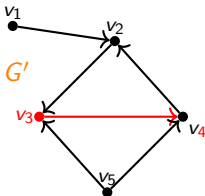


Adjacency matrix of G :

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

- ▶ The row sum of the i -th row equals the column sum of the i -th column.
- ▶ This common row/column sum equals the **degree** $\deg(v_i)$.

Directed case:



Adjacency matrix of G' :

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

- ▶ The row sum of the i -th row equals the **out-degree** $\deg^+(v_i)$.
- ▶ The column sum of the j -th column equals the **in-degree** $\deg^-(v_j)$.

Adjacency list

Definition: Let $G = (V, E)$ be a **simple** graph. Suppose that $V = \{v_1, \dots, v_n\}$. The **adjacency list** of G is an array $A[1..n]$ of length n , whose entries are linked lists, such that:

- ▶ (If G is undirected): $A[i]$ is a linked list whose elements are pointers to vertices v_j incident to v_i .
- ▶ (If G is directed): $A[i]$ is a linked list whose elements are pointers to vertices v_j such that $(v_i, v_j) \in E$.

Note: This array of linked lists can be implemented as a hash table, where each slot is a linked list (i.e. collisions are resolved by chaining).

Possible Python implementation: Python dictionary A with key-value pairs `name_i : list_i`, where `name_i` is a character string representing the name for vertex v_i , and `list_i` is the linked list associated to v_i , which is accessed via $A[\text{name}_i]$.

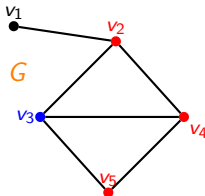
Intuition: Store vertices, and for each vertex v , store those vertices that can be reached from v .

Advantages of adjacency list as a graph representation:

- ▶ Very compact representation when number of edges is small.
- ▶ Many graph algorithms use this graph representation.

Examples of adjacency lists

Undirected case:

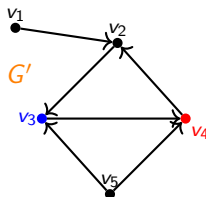


Adjacency list A of G :

$A[v_1]$		→		→	v_2	
$A[v_2]$		→	v_1	→	v_3	→ v_4
$A[v_3]$		→	v_2	→	v_4	→ v_5
$A[v_4]$		→	v_2	→	v_3	→ v_5
$A[v_5]$		→	v_3	→	v_4	

- ▶ Number of elements in linked list $A[i]$ is $\deg(v_i)$.

Directed case:



Adjacency list A of G' :

$A[v_1]$		→	v_2	
$A[v_2]$		→	v_3	
$A[v_3]$		→	v_4	
$A[v_4]$		→	v_2	
$A[v_5]$		→	v_3	→ v_4

- ▶ Number of elements in linked list $A[i]$ is out-degree $\deg^+(v_i)$.

Summary

- ▶ Graph isomorphism
- ▶ Graph invariants
- ▶ Graph representations